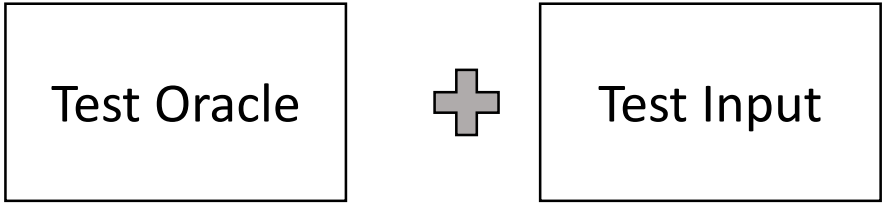
CS3213: Foundations of Software Engineering

Automated Test-case Generation Overview

© Copyright National University of Singapore. All Rights Reserved.

Recap: Test Case

```
@Test
public void testAdd() {
    assertEquals(5, add(2, 3));
}
```

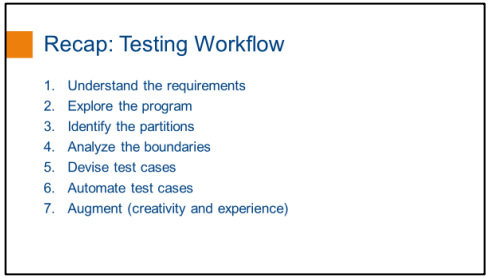


Example-based Testing

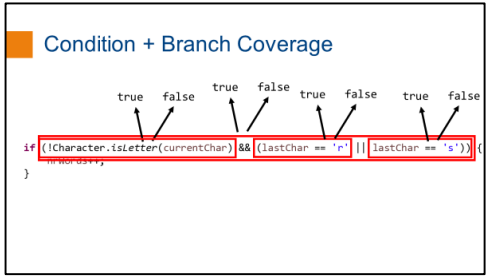
```
@Test
public void testAdd() {
    assertEquals(5, add(2, 3));
}
```

Test Oracle + Test Input

Specification-based Testing



Structural Testing



Automated Test-case Generation

- Automatically construct a test oracle and/or test input

Automated Test-case Generation

- Automatically construct a test oracle and/or test input

https://tenor.com/de/view/guy-long-hair-mustache-magic-shia-labeouf-gif-17741293

Automated Test-case Generation

- Automatically construct a test oracle and/or test input
  - Based on some specifications/insights
  - Test oracle might be incomplete

Automated Test-case Generation: LLMs?

Gemini

Automated Test-case Generation: Why?

- Manually-written, example-based tests might be incomprehensive and miss bugs
- Automated test-case generation approaches are fast and cost-effective
- Low maintenance effort

Black-, Grey-, and White-box Approaches

Black-box: no internal information

- White-box: internal information

Satisfiability Modulo Theories

- SMT solvers: solve a formula based on a given theory
  - For example: theory of Linear Integer Arithmetic
- Input:
  - Example: (a > 5) and (a != 6)
  - Output: model that satisfies the formula
  - Example: a = 7
- Solving such formulas is an NP hard problem, and for many theories undecidable

https://github.com/Z3Prover/z3

White-box Fuzzing: SAGE

- SAGE is a white-box fuzzer that was employed at Microsoft
- Found 1/3 of the bugs discovered by fuzzing during the development of Windows 7
- Start with a concrete input
- Execute the program with this input and gather path constraints along the way
- Systematically negate and solve them with a constraint solver

acmqueue SAGE: Whitebox Fuzzing for Security Testing

SAGE has had a remarkable impact at Microsoft.

Patrice Godefroid, Michael Y. Levin, David Molnar, Microsoft

Most *ACM Queue* readers might think of "program verification research" as mostly theoretical with little impact on the world at large. Think again. If you are reading these lines on a PC running some form of Windows (like 93-plus percent of PC users—that is, more than a billion people), then you have been affected by this line of work—without knowing it, which is precisely the way we want it to be.

THE HIGH COST OF SECURITY BUGS

Every second Tuesday of every month, also known as "Patch Tuesday," Microsoft releases a list of security bulletins and associated security patches to be deployed on hundreds of millions of machines worldwide. Each security bulletin costs Microsoft and its users millions of dollars. If a monthly security update costs you $0.001 (one tenth of one cent) in just electricity or loss of productivity, then this number multiplied by a billion people is $1 million. Of course, if malware were spreading on your machine, possibly leaking some of your private data, then that might cost you much more than $0.001. This is why we strongly encourage you to apply those pesky security updates.

Many security vulnerabilities are a result of programming errors in code for parsing files and packets that are transmitted over the Internet. For example, Microsoft Windows includes parsers for hundreds of file formats.

If you are reading this article on a computer, then the picture shown in figure 1 is displayed on your screen after a jpg parser (typically part of your operating system) has read the image data, decoded it, created new data structures with the decoded data, and passed those to the graphics

KLEE

klee Public

Watch 100 | Fork 714 | Starred 2.9k

master | 15 Branches | 12 Tags | Go to file t | Add file | Code

jbuening and ccadar InstructionInfoTable: add missing cstdint include ✕ efa1e05 · 2 weeks ago 2,817 Commits

.github	Remove workaround for MSan crashes	3 months ago
cmake	find_z3.cmake: Remove old unused code	2 months ago
docs	doxygen.cfg.in: DOXYGEN_OUTPUT_DIR	3 years ago
examples	Use #include "klee/..." (instead of #include <klee/...>) consist...	7 years ago
include/klee	InstructionInfoTable: add missing cstdint include	2 weeks ago
lib	Disallow arrays > 2^32 bytes to become symbolic at a symbo...	3 months ago
runtime	sys/statfs include under linux directly	8 months ago
scripts	Removed support for MSan from build scripts	3 months ago
test	Added test case where an array > 2^32 becomes symbolic	3 months ago
tools	Check for libutil	8 months ago
unittests	Removed annotations about not being able to use MSan for t...	3 months ago

About

KLEE Symbolic Execution Engine

klee-se.org/

symbolic-execution klee

Readme
View license
Activity
Custom properties
2.9k stars
100 watching
714 forks
Report repository

Releases 12

KLEE 3.2 Latest
on Dec 23, 2025

+ 11 releases

https://github.com/klee/klee

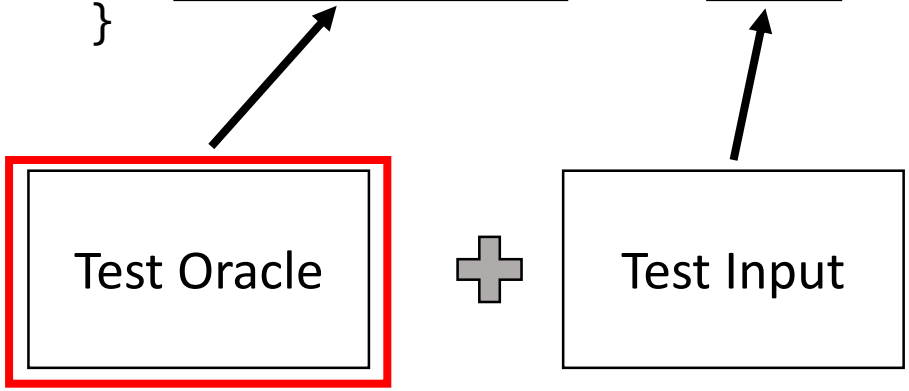
Black-, Grey-, and White-box Approaches

Black-box: no internal information

- White-box: internal information
- Grey-box: some internal information

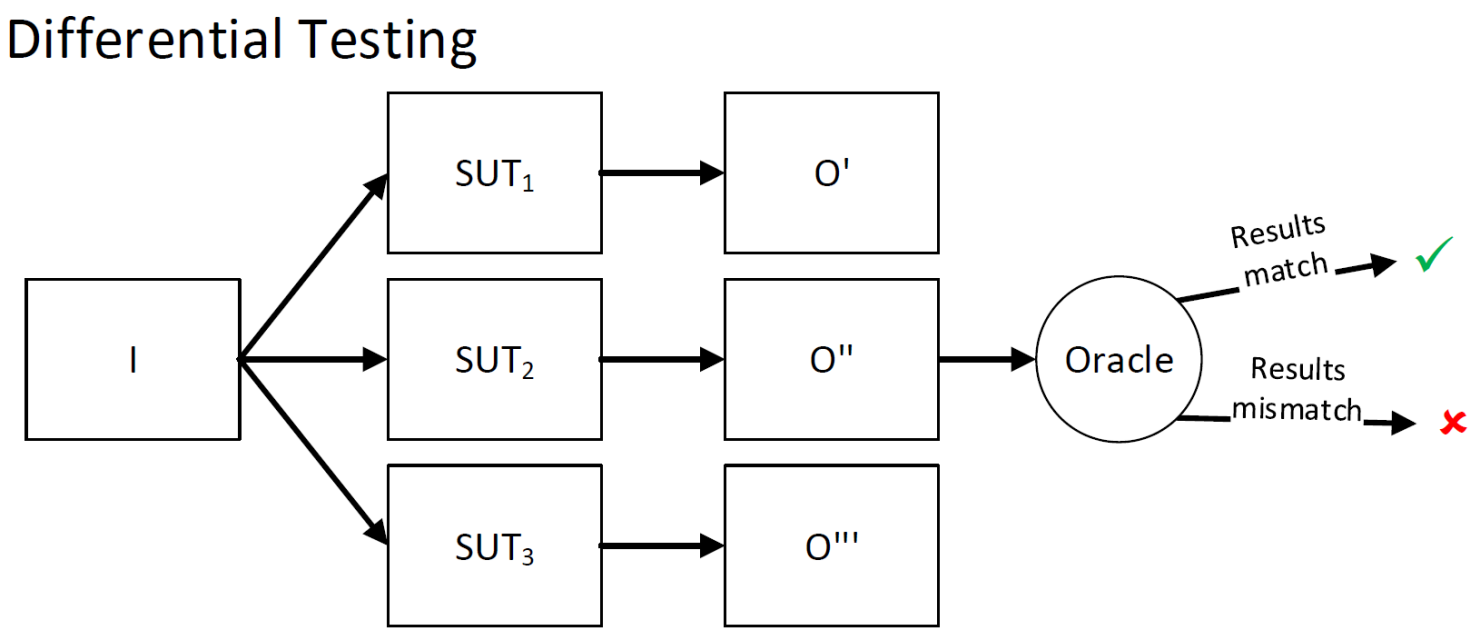
Test Case

```
@Test
public void testAdd() {
    assertEquals(5, add(2, 3));
}
```

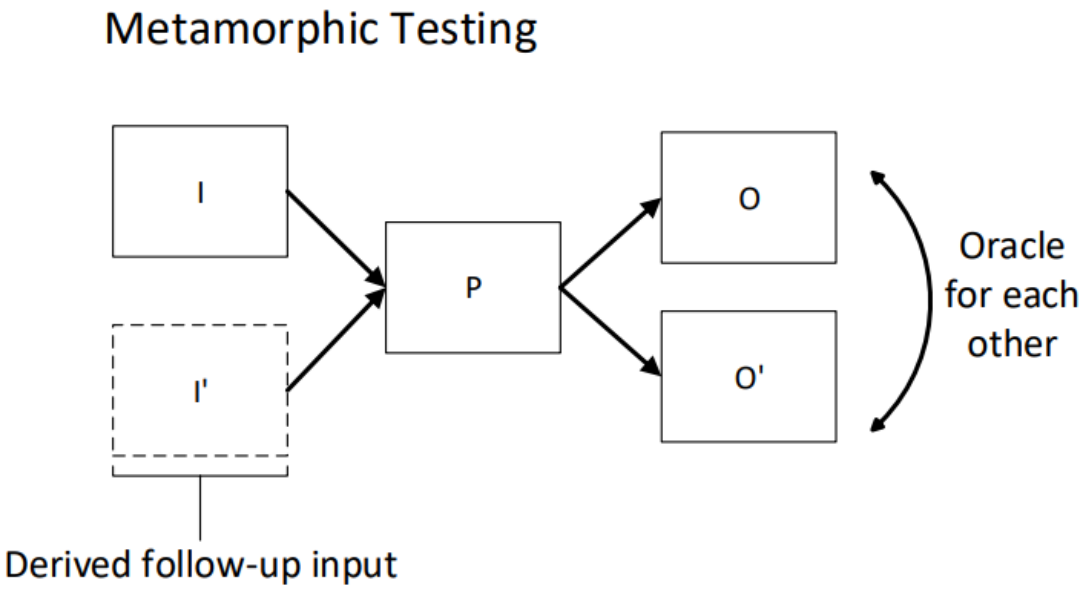


Differential Testing

I: Input
O: Output
SUT: Software Under Test



Metamorphic Testing



Property-based Testing

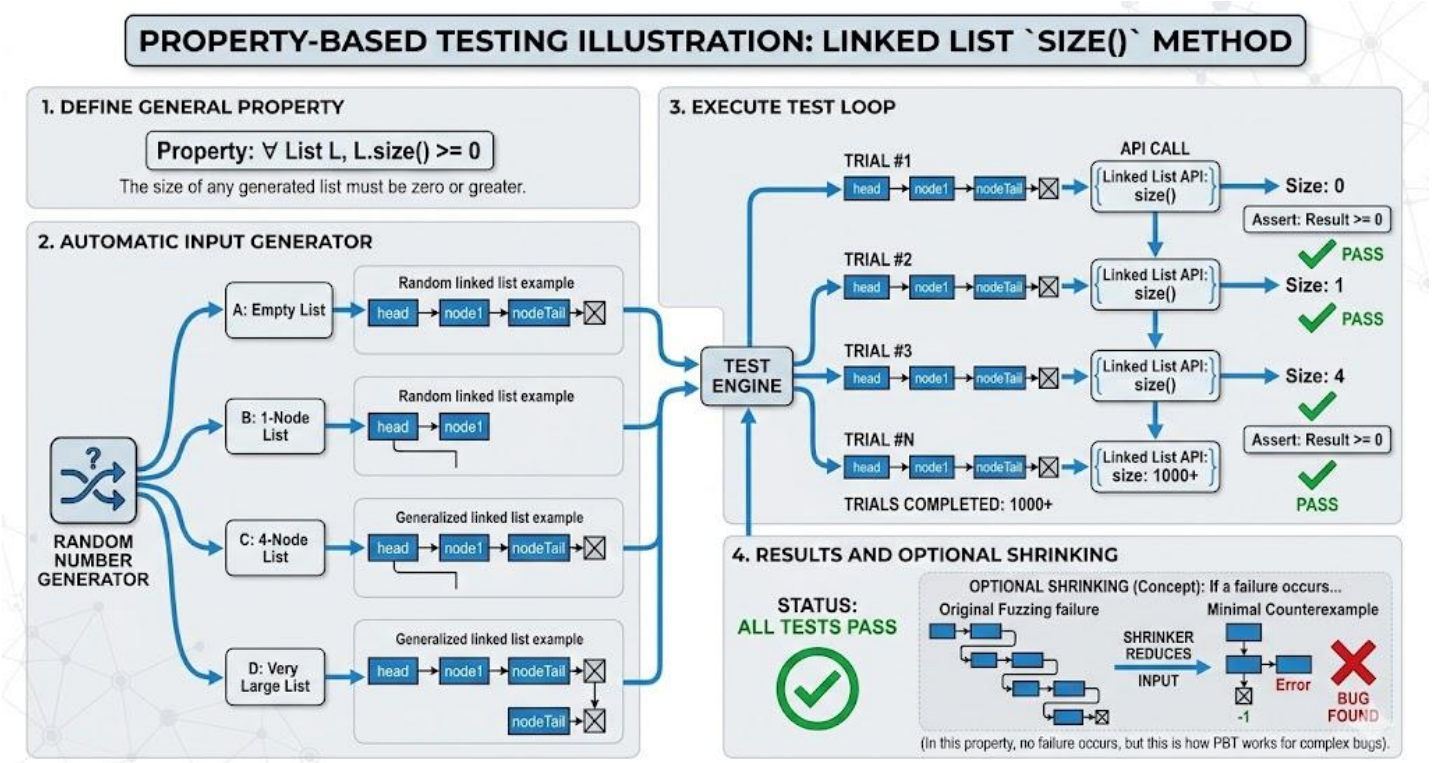
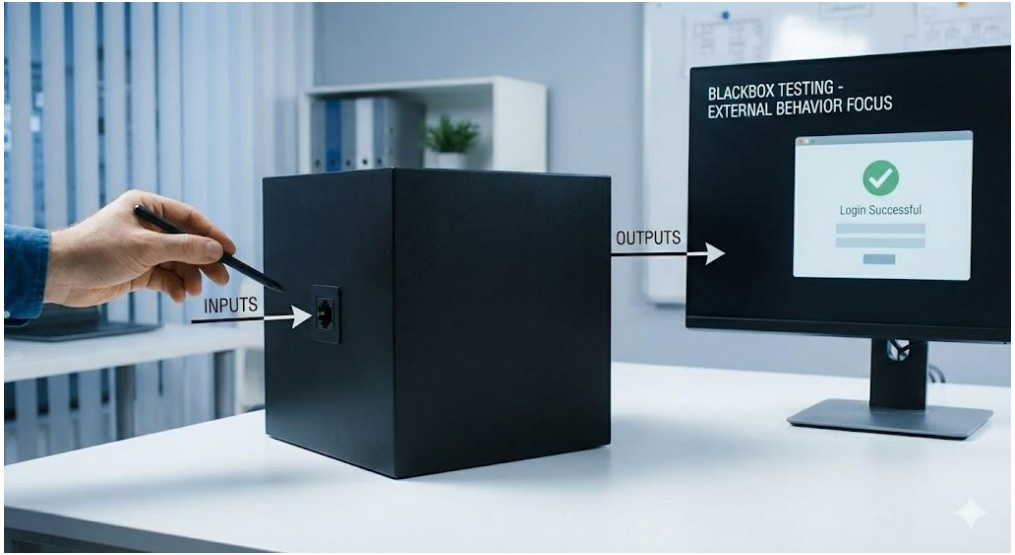


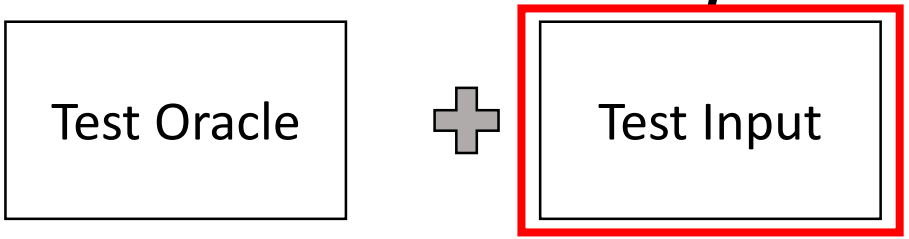
Image by Gemini

Black-box/Opaque-box Testing



Test Case

```
@Test
public void testAdd() {
    assertEquals(5, add(2, 3));
}
```



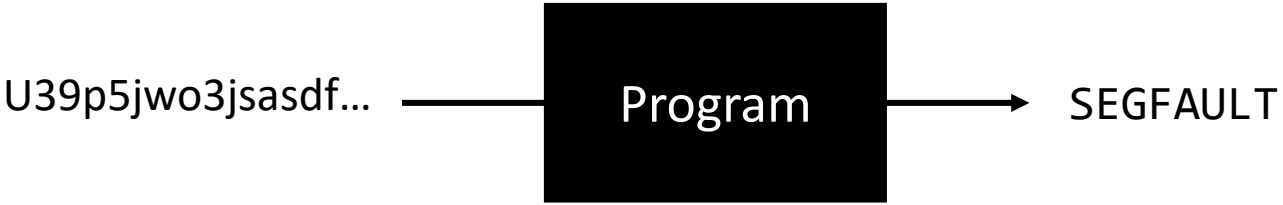
Generation vs Mutation

- Generation-based: generate inputs from scratch
- Mutation-based: start with initial seeds that are mutated

Generation vs Mutation

- Generation-based: generate inputs cases from scratch
- Mutation-based: start with initial seeds that are mutated

(Black-box) Fuzzing



Input Generator: Example

```
import random

# Define the grammar for arithmetic expressions
OPERATORS = ["+", "-", "*", "/"]
NUMBERS = list(range(1, 10))

def generate_expression(depth=0, max_depth=3):
    """Generate a random arithmetic expression as a string."""
    # Base case: just return a number
    if depth >= max_depth or random.random() < 0.3:
        return str(random.choice(NUMBERS))

    # Recursive case: combine two expressions with an operator
    left = generate_expression(depth + 1, max_depth)
    right = generate_expression(depth + 1, max_depth)
    op = random.choice(OPERATORS)

    return f"({left} {op} {right})"
```

Code by ChatGPT

(Naïve) Random Testing: Csmith

https://dl.acm.org/doi/10.1145/1993316.1993532

- Random program generator for C
- Challenge: generate valid programs free of undefined behavior
- Found hundreds of bugs

https://embed.cs.utah.edu/csmith/

Finding and Understanding Bugs in C Compilers

Xuejun Yang Yang Chen Eric Eide John Regehr
University of Utah, School of Computing
{jxyang, chenyang, eeide, regehr}@cs.utah.edu

Abstract

Compilers should be correct. To improve the quality of C compilers, we created Csmith, a randomized test-case generation tool, and spent three years using it to find compiler bugs. During this period we reported more than 325 previously unknown bugs to compiler developers. Every compiler we tested was found to crash and also to silently generate wrong code when presented with valid input. In this paper we present our compiler-testing tool and the results of our bug-hunting study. Our first contribution is to advance the state of the art in compiler testing. Unlike previous tools, Csmith generates programs that cover a large subset of C while avoiding the undefined and unspecified behaviors that would destroy its ability to automatically find wrong-code bugs. Our second contribution is a collection of qualitative and quantitative results about the bugs we have found in open-source C compilers.

Categories and Subject Descriptors D.2.5 [*Software Engineering*]: Testing and Debugging—testing tools; D.3.2 [*Programming Languages*]: Language Classifications—C; D.3.4 [*Programming Languages*]: Processors—compilers

General Terms Languages, Reliability

Keywords compiler testing, compiler defect, automated testing, random testing, random program generation

1. Introduction

The theory of compilation is well developed, and there are compiler frameworks in which many optimizations have been proved correct. Nevertheless, the practical art of compiler construction involves a morass of trade-offs between compilation speed, code quality, code debuggability, compiler modularity, compiler retargetability, and other goals. It should be no surprise that optimizing compilers—like all complex software systems—contain bugs.

Miscompilations often happen because optimization safety checks are inadequate, static analyses are unsound, or transformations are flawed. These bugs are out of reach for current and future automated program-verification tools because the specifications that need to be checked were never written down in a precise way, if they were written down at all. Where verification is impractical, however, other methods for improving compiler quality can succeed. This paper reports our experience in using testing to make C compilers better.

```
1  int foo (void) {
2    signed char x = 1;
3    unsigned char y = 255;
4    return x > y;
5  }
```

Figure 1. We found a bug in the version of GCC that shipped with Ubuntu Linux 8.04.1 for x86. At all optimization levels it compiles this function to return 1; the correct result is 0. The Ubuntu compiler was heavily patched; the base version of GCC did not have this bug.

We created Csmith, a randomized test-case generator that supports compiler bug-hunting using differential testing. Csmith generates a C program; a test harness then compiles the program using several compilers, runs the executables, and compares the outputs. Although this compiler-testing approach has been used before [6, 16, 23], Csmith's test-generation techniques substantially advance the state of the art by generating random programs that are expressive—containing complex code using many C language features—while also ensuring that every generated program has a single interpretation. To have a unique interpretation, a program must not execute any of the 191 kinds of undefined behavior, nor depend on any of the 52 kinds of unspecified behavior, that are described in the C99 standard.

For the past three years, we have used Csmith to discover bugs in C compilers. Our results are perhaps surprising in their extent: to date, we have found and reported more than 325 bugs in mainstream C compilers including GCC, LLVM, and commercial tools. Figure 1 shows a representative example. Every compiler that we have tested, including several that are routinely used to compile safety-critical embedded systems, has been crashed and also shown to silently miscompile valid inputs. As measured by the responses to our bug reports, the defects discovered by Csmith are important. Most of the bugs we have reported against GCC and LLVM have been fixed. Twenty-five of our reported GCC bugs have been classified as P1, the maximum, release-blocking priority for GCC defects. Our results suggest that fixed test suites—the main way that compilers are tested—are an inadequate mechanism for quality control.

We claim that Csmith is an effective bug-finding tool in part because it generates tests that explore atypical combinations of C language features. Atypical code is *not* unimportant code, however; it is simply underrepresented in fixed compiler test suites. Developers who stray outside the well-tested paths that represent a compiler's "comfort zone"—for example by writing kernel code or embedded systems code, using esoteric compiler options, or automatically generating code—can encounter bugs quite frequently.

Grammar-based Fuzzing

- Idea: use a grammar, which specifies the syntactic structure of a language
  - **Terminals**: The actual symbols that appear in the final string generated by the grammar
  - **Non-terminals**: abstract placeholders
  - **Production rule**: how a non-terminal can be replaced with terminals or other non-terminals
- The fuzzer randomly expands all production rules from the root until no more non-terminals remain

Program Generator: Example

```
import random

# Define the grammar for arithmetic expressions
OPERATORS = ["+", "-", "*", "/"]
NUMBERS = list(range(1, 10))

def generate_expression(depth=0, max_depth=3):
    """Generate a random arithmetic expression as a string."""
    # Base case: just return a number
    if depth >= max_depth or random.random() < 0.3:
        return str(random.choice(NUMBERS))

    # Recursive case: combine two expressions with an operator
    left = generate_expression(depth + 1, max_depth)
    right = generate_expression(depth + 1, max_depth)
    op = random.choice(OPERATORS)

    return f"({left} {op} {right})"
```

Code by ChatGPT

Grammar-Based Fuzzing

Context-Free Grammar (CFG)

Production rule

Non-terminal

```
<expr>   ::= <term> | <expr> "+" <term> | <expr> "-" <term>
<term>   ::= <factor> | <term> "*" <factor> | <term> "/" <factor>
<factor> ::= [illegible] <expr> ")"
<number> ::= [illegible] <digit> <number>
<digit>  ::= "0" | "1" | ... | "9"
```

Terminal

Grammar-Based Fuzzing

```
<expr>   ::= <term> | <expr> "+" <term> | <expr> "-" <term>
<term>   ::= <factor> | <term> "*" <factor> | <term> "/" <factor>
<factor> ::= <number> | "(" <expr> ")"
<number> ::= <digit> | <digit> <number>
<digit>  ::= "0" | "1" | ... | "9"
```

Grammar-Based Fuzzing

```
<expr>

<expr>   ::= <term> | <expr> "+" <term> | <expr> "-" <term>
<term>   ::= <factor> | <term> "*" <factor> | <term> "/" <factor>
<factor> ::= <number> | "(" <expr> ")"
<number> ::= <digit> | <digit> <number>
<digit>  ::= "0" | "1" | ... | "9"
```

Grammar-Based Fuzzing

```
<term>

<expr>   ::= <term> | <expr> "+" <term> | <expr> "-" <term>
<term>   ::= <factor> | <term> "*" <factor> | <term> "/" <factor>
<factor> ::= <number> | "(" <expr> ")"
<number> ::= <digit> | <digit> <number>
<digit>  ::= "0" | "1" | ... | "9"
```

Grammar-Based Fuzzing

```
<term>

<expr>   ::= <term> | <expr> "+" <term> | <expr> "-" <term>
<term>   ::= <factor> | <term> "*" <factor> | <term> "/" <factor>
<factor> ::= <number> | "(" <expr> ")"
<number> ::= <digit> | <digit> <number>
<digit>  ::= "0" | "1" | ... | "9"
```

Grammar-Based Fuzzing

```
<term> * <factor>
```

```
<expr>   ::= <term> | <expr> "+" <term> | <expr> "-" <term>
<term>   ::= <factor> | <term> "*" <factor> | <term> "/" <factor>
<factor> ::= <number> | "(" <expr> ")"
<number> ::= <digit> | <digit> <number>
<digit>  ::= "0" | "1" | ... | "9"
```

Grammar-Based Fuzzing

```
<term> * <factor>
```

```
<expr>   ::= <term> | <expr> "+" <term> | <expr> "-" <term>
<term>   ::= <factor> | <term> "*" <factor> | <term> "/" <factor>
<factor> ::= <number> | "(" <expr> ")"
<number> ::= <digit> | <digit> <number>
<digit>  ::= "0" | "1" | ... | "9"
```

Grammar-Based Fuzzing

```
0 * <factor>
```

```
<expr>   ::= <term> | <expr> "+" <term> | <expr> "-" <term>
<term>   ::= <factor> | <term> "*" <factor> | <term> "/" <factor>
<factor> ::= <number> | "(" <expr> ")"
<number> ::= <digit> | <digit> <number>
<digit>  ::= "0" | "1" | ... | "9"
```

Grammar-Based Fuzzing

```
0 * <factor>
```

```
<expr>   ::= <term> | <expr> "+" <term> | <expr> "-" <term>
<term>   ::= <factor> | <term> "*" <factor> | <term> "/" <factor>
<factor> ::= <number> | "(" <expr> ")"
<number> ::= <digit> | <digit> <number>
<digit>  ::= "0" | "1" | ... | "9"
```

Grammar-Based Fuzzing

```
0 * 19
```

```
<expr>   ::= <term> | <expr> "+" <term> | <expr> "-" <term>
<term>   ::= <factor> | <term> "*" <factor> | <term> "/" <factor>
<factor> ::= <number> | "(" <expr> ")"
<number> ::= <digit> | <digit> <number>
<digit>  ::= "0" | "1" | ... | "9"
```

ANTLR Grammars

```
deallocatestmt
    : DEALLOCATE name
    | DEALLOCATE PREPARE name
    | DEALLOCATE ALL
    | DEALLOCATE PREPARE ALL
    ;

insertstmt
    : opt_with_clause INSERT INTO insert_target insert_rest opt_on_conflict returning_clause
    ;

insert_target
    : qualified_name (AS colid)?
    ;

insert_rest
    : selectstmt
    | OVERRIDING override_kind VALUE_P selectstmt
    | OPEN_PAREN insert_column_list CLOSE_PAREN (OVERRIDING override_kind VALUE_P)? selectstmt
    | DEFAULT VALUES
    ;
```

https://github.com/antlr/grammars-v4/blob/master/sql/postgresql/PostgreSQLParser.g4

Generation vs. Mutation

- Generation-based: generate inputs/test cases from scratch
- Mutation-based: start with initial seeds that are mutated

Black-box Input Generation

- Advantage: simple and efficient test-case generation
- Disadvantage: unlikely to uncover deep bugs, especially when the input format is complex (e.g., image parsers)

U39p5jwo3jsasdf…

```
if (0x89504E470D0A1A0A != magicNumberField)
{
    error("invalid png file");
}
```

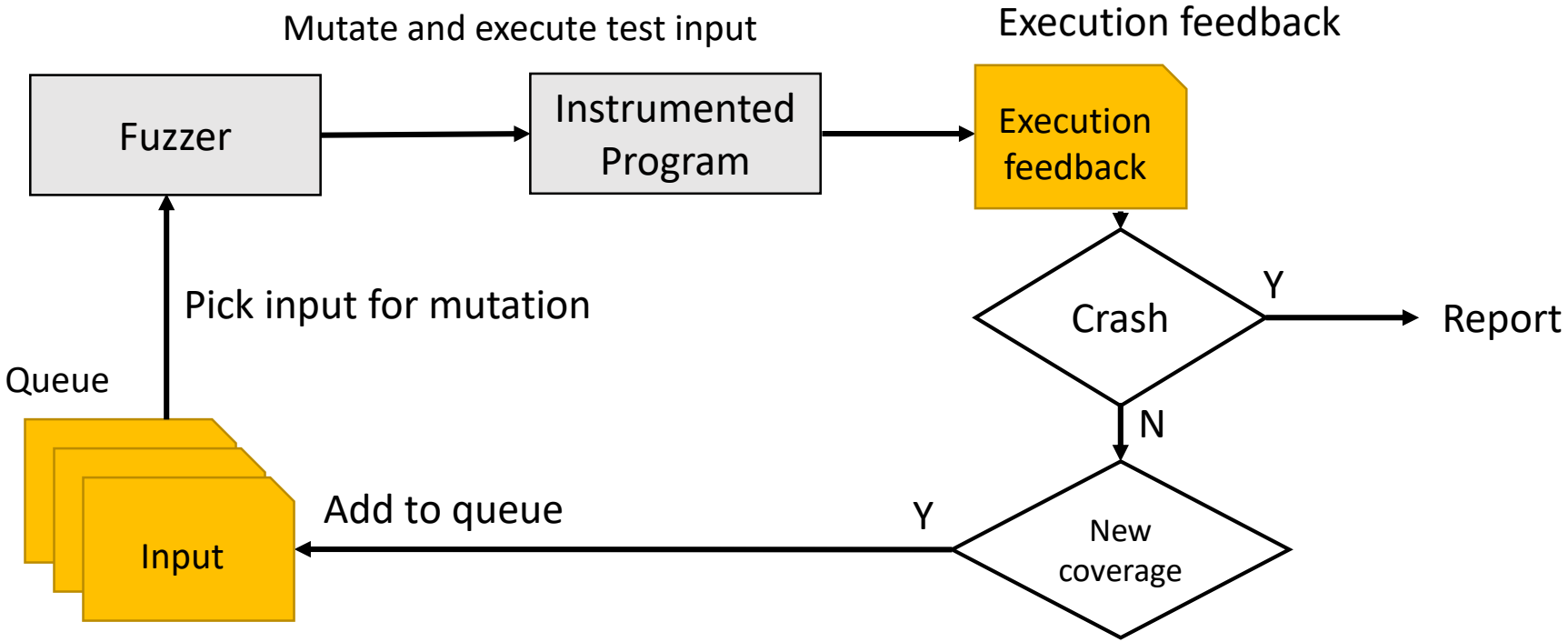
Invalid
png file

Mutation-based Fuzzing

- Rather than passing random input to the program, mutate existing inputs
- Advantage: higher-quality inputs (assuming a diverse input *seed corpus*)
- Disadvantage: requires seed inputs to mutate

<h2>header</h2> $\xrightarrow{\text{mutate}}$ <h2>header<<</h2>

Greybox Fuzzers: Simplified Scheme



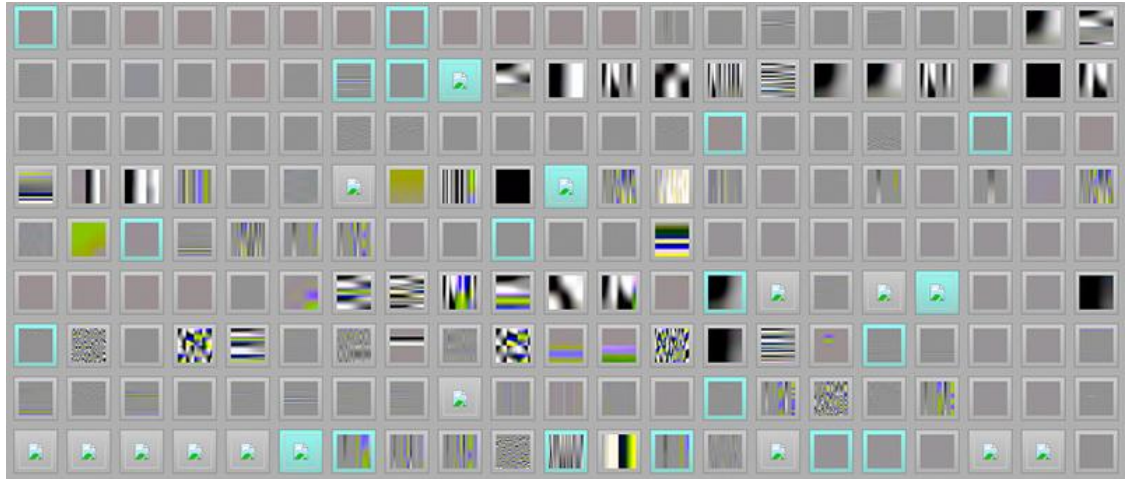
Grey-box Fuzzing

- American Fuzzy Lop (AFL)
  - Lop, not loop
- Lightweight instrumentation
- Puts more energy into further mutating inputs that cover new code paths

https://github.com/AFLplusplus/AFLplusplus

Grey-box Fuzzing

- Starting from a “hello” text file, greybox fuzzing can generate valid images within 6 hours



https://lcamtuf.blogspot.com/2014/11/pulling-jpegs-out-of-thin-air.html

AFL++

AFLplusplus

Search

Site map

- Features
- Build & Install
- Documentation
- Tutorials
- Papers
- LibAFL Book

Downloads

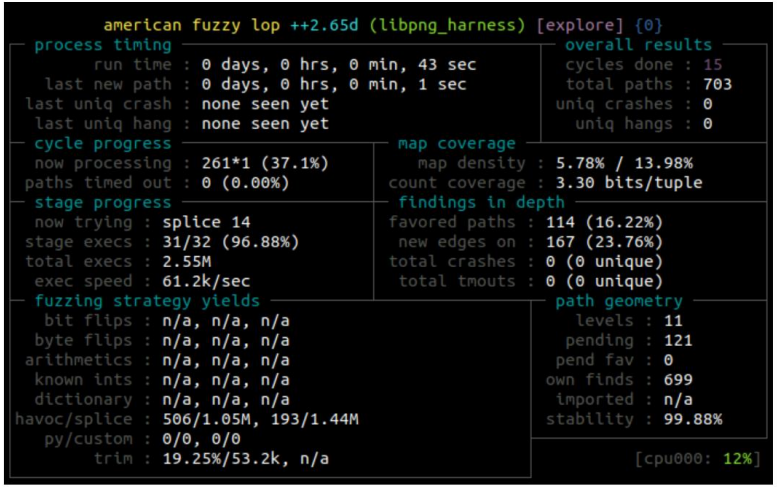
- Release 4.09c
- All releases
- Current devel
- License

Links

- Repo (GitHub)
- Donations
- Mailing list

AFL++ Overview

AFLplusplus is the daughter of the American Fuzzy Lop fuzzer by Michał "lcamtuf" Zalewski and was created initially to incorporate all the best features developed in the years for the fuzzers in the AFL family and not merged in AFL cause it is not updated since November 2017.



AFL++: Combining Incremental Steps of Fuzzing Research

Andrea Fioraldi[†], Dominik Maier[‡], Heiko Eißfeldt, Marc Heuse[§]
{andrea, dominik, heiko, marc}@aflplus.plus
†Sapienza University of Rome, ‡TU Berlin, §The Hacker's Choice

Abstract

In this paper, we present AFL++, a community-driven open-source tool that incorporates state-of-the-art fuzzing research, to make the research comparable, reproducible, combinable and — most importantly – useable. It offers a variety of novel features, for example its *Custom Mutator API*, able to extend the fuzzing process at many stages. With it, mutators for specific targets can also be written by experienced security testers. We hope for AFL++ to become a new baseline tool not only for current, but also for future research, as it allows to test new techniques quickly, and evaluate not only the effectiveness of the single technique versus the state-of-the-art, but also in combination with other techniques. The paper gives an evaluation of hand-picked fuzzing technologies — shining light on the fact that while each novel fuzzing method can increase performance in some targets — it decreases performance for other targets. This is an insight future fuzzing research should consider in their evaluations.

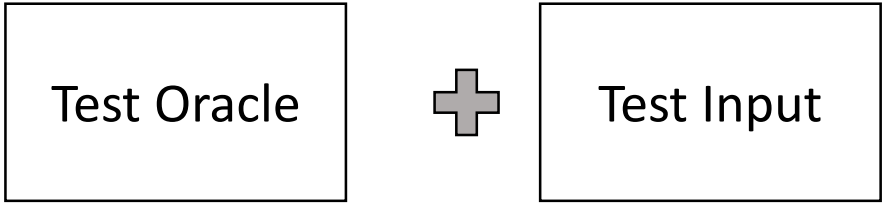
to combine functionality with the compatible techniques that address different, but related problems in fuzzing — for example picking a recent seed scheduling for their mutator. A new feedback concept may not live up to its full potential if it cannot be combined with existing techniques solving other problems — like overcoming hard comparison instructions — reducing the impact of the research on paper due to lackluster statistics.

In this paper, we try to solve these problems by raising the bar of broadly available, research-backed, fuzzing, and by giving researchers an extensible API to build upon. We propose a novel fuzzing framework, AFL++. Future research can use AFL++ as a new baseline. It gives researchers the possibility to evaluate combinations of their proposals with state-of-the-art orthogonal features already implemented in AFL++ — with a highly reduced implementation effort. At the same time, it offers industry professionals a large range of easy-to-use features adapted from cutting-edge research, that can greatly improve the outcome of a fuzzing campaign.

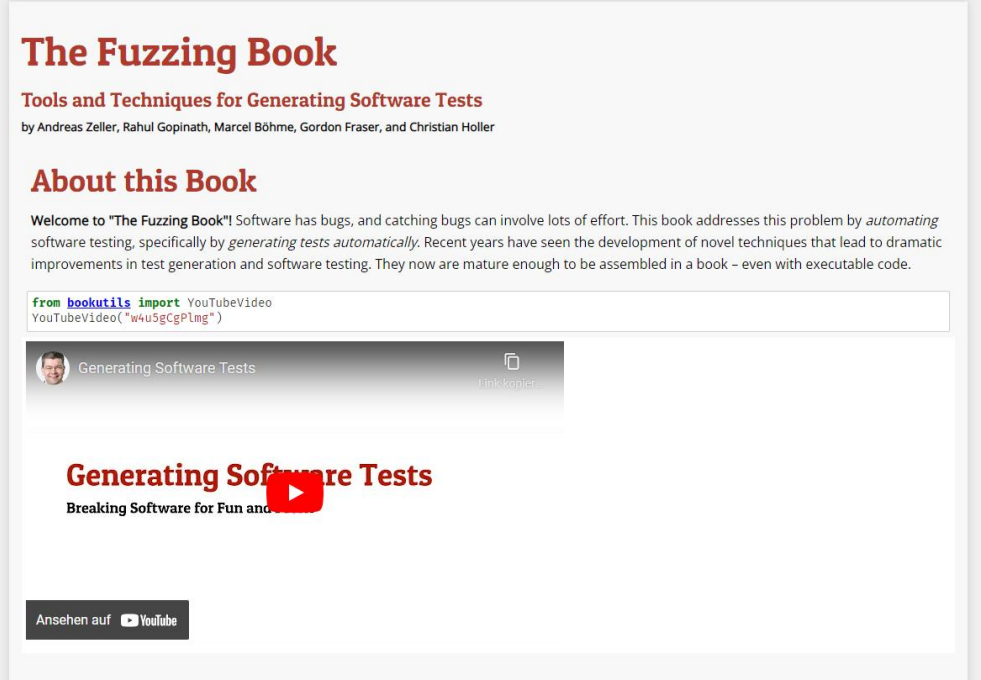
https://aflplus.plus/

Test Case

```
@Test
public void testAdd() {
    assertEquals(5, add(2, 3));
}
```



Fuzzing: Additional Resources



https://www.fuzzingbook.org/

Summary and Takeaways

- Automated test case generation vs. example-based testing
- Black-box, white-box, and grey-box approaches
- Generation-based: naïve black-box fuzzing, input generators, and grammar-based fuzzing
- Mutation-based: fuzzing, grey-box fuzzing